# Opentron Control — Pre-Implementation Analysis

**Status:** Analysis only. NO code or data has been changed. **Date:** 2026-04-16 **Branch:** `wax-and-reagent-split-up`

---

## 1. The change in one paragraph

Today, wax filling and reagent filling each occupy a single "robot session" from setup through fridge storage. The OT-2 robot completes its actual run partway through, but the run document stays in an **active** state for the operator's manual post-processing (cooling/QC/storage for wax; inspection/top-sealing/storage for reagent). That keeps the robot logically locked even though it's physically idle. The change: split each workflow at the point the OT-2 finishes. The run is "complete" from the robot's perspective at that boundary, freeing the robot for a new run. The post-processing steps move to a new top-level page called **Opentron Control**, organized as queues of cartridges-awaiting-action, accessed independently of any robot.

Crucially, **all current functionality, mongo writes, equipment hooks, audit logs, admin overrides, test buttons, and timers stay**. They just live behind a different navigation entry point.

---

## 2. Current state — split points identified

### Wax filling (6 steps)

| # | DB Status | UI Stage | Component | Robot needed? |
|---|---|---|---|---|
| 1 | `Setup` | Setup | `SetupConfirmation.svelte` | Idle (operator confirms) |
| 2 | `Loading` | Loading | `WaxPreparation.svelte` + `DeckLoadingGrid.svelte` | Idle (operator preps) |
| 3 | `Running` | Running | `RunExecution.svelte` | **Yes — OT-2 actively runs** |
| **— SPLIT POINT —** | | | | |
| 4 | `Awaiting Removal` | Awaiting Removal | `PostRunCooling.svelte` | No (deck removed, cartridges in oven) |
| 5 | `QC` | QC | `QCInspection.svelte` | No (cartridges cooling) |
| 6 | `Storage` | Storage | `CompletionStorage.svelte` | No (assigning fridge locations) |
| terminal | `completed` | — | — | — |

**Current robot lock:** `WaxFillingRun.find({ 'robot._id': X, status: { $in: ACTIVE_STAGES } })` where `ACTIVE_STAGES` = all 6 stages. Reference: `src/routes/manufacturing/wax-filling/+page.server.ts:50-51` and `+layout.server.ts:9-10`.

**Transition action at split point:** `confirmDeckRemoved` ( `+page.server.ts:615-652` ) writes `status: 'Awaiting Removal'` and writes `ovenCure.entryTime` on each cartridge. **This is the action where the OT-2 is**

**physically free.**

### Reagent filling (6 steps)

| # | DB Status | UI Stage | Component | Robot needed? |
|---|---|---|---|---|
| 1 | `Setup` | Setup | `SetupConfirmation.svelte` | Idle |
| 2 | `Loading` | Loading | `DeckLoadingGrid.svelte` + `ReagentPreparation.svelte` | Idle |
| 3 | `Running` | Running | `RunExecution.svelte` | **Yes — OT-2 fills reagents** |
| **— SPLIT POINT —** | | | | |
| 4 | `Inspection` | Inspection | `Inspection.svelte` | No |
| 5 | `Top Sealing` | Top Sealing | `TopSealing.svelte` | No (operator seals at workstation) |
| 6 | `Storage` | Storage | `CompletionStorage.svelte` | No |
| terminal | `Completed` | — | — | — |

**Current robot lock:** `ReagentBatchRecord.findOne({ 'robot._id': X, status: { $nin: TERMINAL } })` where TERMINAL is only the end states. So statuses 1–6 all lock the robot. Reference: `+layout.server.ts:88-91` and `+page.server.ts:180-190`.

**Transition action at split point:** `completeRunFilling` ( `+page.server.ts:439-503` ) writes `status: 'Inspection'` and writes the `reagentFilling` subdoc on each cartridge. **This is the action where the OT-2 is physically free.**

### Important: the dashboard already knows about "deck free" stages

`src/routes/manufacturing/+page.server.ts:149-152` (added in the recent dev commit `58eb1c3` ) already classifies post-OT-2 stages as `WAX_DECK_FREE` / `REAGENT_DECK_FREE` . The dashboard *displays* this correctly today. What's missing is that the **run-lock query still includes those statuses**, so even though the dashboard says "Robot Free", the operator still can't start a new wax/reagent run on that robot. That's the actual block.

---

## 3. The proposed restructure

### 3.1 New nav entry: `/manufacturing/opentron-control`

A single landing page that surfaces three things at once:

```
┌──────────────────────────────────────────────────────────┐
│  Opentron Control                                        │
├──────────────────────────────────────────────────────────┤
│  Robots                                                  │
│  ┌─────────────────┐  ┌─────────────────┐               │
│  │ OT-2 Left       │  │ OT-2 Right      │ ...           │
│  │ Available       │  │ Running Wax     │               │
```

```
│   │ [Start Wax] [Start Reagent]          │                │
│   └──────────────────┘  └──────────────────┘                │
├──────────────────────────────────────────────────────────────┤
│  Wax Cartridges Requiring Inspection & Storage              │
│  • Run abc123  — 24 cartridges  — In oven 12 min            │
│  • Run def456  — 18 cartridges  — Ready (QC blocked,        │
│                                   cooling timer active)     │
├──────────────────────────────────────────────────────────────┤
│  Reagent Cartridges Requiring Top Sealing & Storage         │
│  • Run xyz789  — 24 cartridges  — Sealed 4/24               │
│  • Run uvw012  — 24 cartridges  — Awaiting inspection       │
└──────────────────────────────────────────────────────────────┘
```

Clicking a wax run takes the operator to the existing `PostRunCooling → QCInspection → CompletionStorage` flow but in a "detached" mode (no robot context). Clicking a reagent run takes them to the existing `Inspection → TopSealing → CompletionStorage` flow.

### 3.2 New routes

- `GET /manufacturing/opentron-control` — landing (robots + both queues)
- `GET /manufacturing/opentron-control/wax/[runId]` — post-OT-2 wax flow (steps 4-6)
- `GET /manufacturing/opentron-control/reagent/[runId]` — post-OT-2 reagent flow (steps 4-6)

The existing `/manufacturing/wax-filling` and `/manufacturing/reagent-filling` routes stay, but they now **only host steps 1-3** (setup → loading → running). Once step 3 ends, the user is redirected to Opentron Control (or the wax/reagent page goes empty for that robot).

### 3.3 Run-lock semantics change

The run document needs a way to express "the OT-2 is done with this run, but post-processing is still pending." Two equivalent technical approaches:

**Option A — Introduce new status value(s).**

- Wax: add a new status like `post_run` (or `awaiting_qc`). The `confirmDeckRemoved` action writes `post_run` *instead of* `Awaiting Removal`. The UI's `toStage()` mapping function translates `post_run` back to "Awaiting Removal" for display.
- Reagent: same idea — add `post_run`. The `completeRunFilling` action writes it instead of `Inspection`.
- Then "is the robot free?" becomes "is the status equal to `post_run` or any later post-OT-2 status?"
- **Pro:** the status field directly encodes "robot free" without needing a second field.
- **Con:** every place that lists, filters, displays, or maps statuses has to be updated. The status enum widens. The dashboard's existing `WAX_DECK_FREE` constants need to know about the new value. Reports that group by status get a new bucket. Audit logs that record `status: 'Awaiting Removal'` historically would now record `status: 'post_run'` — historical comparisons get awkward.

**Option B — Add a `robotReleasedAt` timestamp field.**

- Don't change any status values. The wax run still goes `Setup → Loading → Running → Awaiting Removal → QC → Storage → completed` exactly as today. Reagent runs are unchanged too.
- Just add one new field to both schemas: `robotReleasedAt: Date`. Set it at the moment the OT-2 finishes (in `confirmDeckRemoved` for wax, in `completeRunFilling` for reagent).
- The "is the robot free?" question becomes a simple field check: `if (run.robotReleasedAt) → robot is free`.

- The queries that gate "can I start a new run on this robot?" change from `{ status: { $in: ACTIVE_STAGES } }` to `{ status: { $in: ACTIVE_STAGES }, robotReleasedAt: { $exists: false } }`. Same set of statuses, just an extra filter.
- **Pro:** minimal blast radius. No status enum changes. UI mappings, dashboards, reports, audit logs all keep working unchanged. The post-OT-2 components don't need to know anything new — they're rendered the same way they are today, just from a different route.
- **Con:** "is the robot free?" requires checking two things (status + flag) instead of one — slightly less ergonomic to read in code.

**Recommendation: Option B.** The smaller blast radius wins. Six query files change in one specific way; everything else stays identical.

### 3.4 Queue lists

Each queue is just a database query already implementable today:

**Wax queue** (cartridges awaiting inspection/storage):

```
WaxFillingRun.find({
  status: { $in: ['Awaiting Removal', 'QC', 'Storage', /* lowercase variants */] },
  // After option B: robotReleasedAt: { $exists: true }, status: { $nin: terminal }
})
```

**Reagent queue:**

```
ReagentBatchRecord.find({
  status: { $in: ['Inspection', 'Top Sealing', 'Storage'] }
})
```

Each row needs: runId, cartridge count, current stage, time-since-OT-2-finish, any pending timer (cooling deadline, etc).

---

## 4. What moves where — file-by-file map

### 4.1 Components — move (no edit)

These `.svelte` components are frozen by the project rules. They get **referenced** from new routes; the files themselves aren't modified.

| Component | From | To (also referenced by) |
|---|---|---|
| `PostRunCooling.svelte` | `wax-filling/+page.svelte` | `opentron-control/wax/[runId]/+page.svelte` |
| `QCInspection.svelte` | `wax-filling/+page.svelte` | `opentron-control/wax/[runId]/+page.svelte` |
| `CompletionStorage.svelte` (wax) | `wax-filling/+page.svelte` | `opentron-control/wax/[runId]/+page.svelte` |
| `Inspection.svelte` | `reagent-filling/+page.svelte` | `opentron-control/reagent/[runId]/+page.svelte` |
| `TopSealing.svelte` | `reagent-filling/+page.svelte` | `opentron-control/reagent/[runId]/+page.svelte` |

| | | |
|---|---|---|
| `CompletionStorage.svelte` (reagent) | `reagent-filling/+page.svelte` | `opentron-control/reagent/[runId]/+page.svelte` |

Since `+page.svelte` files are also frozen, the existing `wax-filling/+page.svelte` and `reagent-filling/+page.svelte` continue to render all 6 steps. After the run hits the split point, the user is **navigated away** to Opentron Control. The "frozen" UI files don't need to learn anything new; they just stop being entered for steps 4-6.

### 4.2 Server actions — keep, but move where they're routed from

Each post-split server action stays exactly as-is in terms of logic. They get exposed from the new `opentron-control/wax/[runId]/+page.server.ts` and `opentron-control/reagent/[runId]/+page.server.ts` as well. The cleanest implementation reuses them via re-export:

**Wax — actions to expose from new route:**

- `confirmDeckRemoved` (already at split point — actually triggers it)
- `confirmCooling`
- `completeQC`
- `rejectCartridge`
- `recordBatchStorage`
- `completeRun`

**Wax — actions explicitly NOT exposed on new route** (per §7.3 decision):

- `cancelRun` / `abortRun` — once the OT-2 finishes successfully, the run cannot be cancelled or aborted. These remain available only on `/manufacturing/wax-filling` for steps 1-3. The actions themselves get a server-side guard rejecting any call where `robotReleasedAt` is set.

**Reagent — actions to expose:**

- `completeRunFilling` (the split-point action)
- `completeInspectionBatch` / `completeInspection`
- `createTopSealBatch`, `scanCartridgeForSeal`, `completeSealBatch`
- `rejectAtSeal` (rejects an individual cartridge during sealing — not a run-level abort)
- `transitionToStorage`
- `recordBatchStorage`
- `completeRun`

**Reagent — actions explicitly NOT exposed on new route** (per §7.3 decision):

- `cancelRun` / `abortRun` — same rule as wax. Only available during steps 1-3 on `/manufacturing/reagent-filling`.

**No mongo write logic changes. No audit log changes. No part-consumption changes.** Every action keeps its current write-once guards, equipment validations, and timers.

### 4.3 The split-point actions need one tiny addition

Just one new field write to free the robot, per Option B:

- `confirmDeckRemoved` (wax): add `robotReleasedAt: now` to the `$set`.
- `completeRunFilling` (reagent): add `robotReleasedAt: now` to the `$set`.

That's the entire mechanism for releasing the robot earlier.

### 4.4 The active-run queries must change

Three places:

| File | Current query | After |
|---|---|---|
| `wax-filling/+page.server.ts:107-108` | `WaxFillingRun.findOne({ 'robot._id': robotId, status: { $in: ACTIVE_STAGES } })` | Add `robotReleasedAt: { $exists: false }` to the filter |
| `wax-filling/+layout.server.ts:9-10` (and the find call below) | Same shape | Same change |
| `reagent-filling/+page.server.ts:180-190` (the wax-blocking check) | `WaxFillingRun.findOne({ 'robot._id': robotId, status: {...non-terminal...} })` | Add `robotReleasedAt: { $exists: false }` |
| `reagent-filling/+page.server.ts:88-91` (the active-reagent query) | `ReagentBatchRecord.findOne({ 'robot._id': robotId, status: { $nin: TERMINAL } })` | Add `robotReleasedAt: { $exists: false }` |
| `reagent-filling/+layout.server.ts:88-91` | Same | Same change |
| `wax-filling/+page.server.ts:106-115` (the reagent-blocking check) | `ReagentBatchRecord.findOne(...)` | Add `robotReleasedAt: { $exists: false }` |
| `manufacturing/+page.server.ts:31-38` (dashboard active-runs query) | `find({ status: { $nin: ['completed', ...] } })` | **Leave alone** — the dashboard wants to show post-OT-2 runs on a robot card; that's its current correct behavior. |

### 4.5 Timers and gates — preserved as-is

| Timer / gate | Where | New location? |
|---|---|---|
| Wax oven minimum (60 min) `minOvenTimeMin` | enforced server-side in `scanBackingLot` (step 2) | unchanged — still in wax-filling step 2 |
| Wax cooling 10 min before QC | enforced server-side in `completeQC` + UI countdown in `QCInspection.svelte:5` | **moves with the QC step** to Opentron Control wax flow. Same code, same gate, same admin bypass via `/api/dev/validate-equipment?type=admin-password`. |
| Wax `coolingWarningMin` (transfer-overdue alarm) | UI in `PostRunCooling.svelte` | moves with PostRunCooling component |
| Wax `removeDeckWarningMin` | UI in `RunExecution.svelte` | stays in wax-filling step 3 |
| Reagent run timer (`fillTimePerCartridgeMin × cartridgeCount`) | computed in `startRun`, displayed in `RunExecution.svelte` | stays in reagent step 3 |

| | | |
|---|---|---|
| Reagent cooling minimum (`minCoolingTimeMin`) | computed in `cooling-queue/+page.server.ts` | already a separate page; can stay there or fold into Opentron Control queue rows |
| **Top-seal deadline (the user mentioned)** | **does not exist today** | see §7 — this needs spec |

### 4.6 Equipment, parts, audit — unchanged

Every equipment lookup ( `/api/dev/validate-equipment` for deck/tray/fridge/cartridge), every `recordTransaction` for inventory, every `AuditLog.create` , every admin override (cooling bypass, oven time override, force-advance) lives inside the **server actions** above. Since the actions don't move (just become callable from a different route), all of this is preserved automatically.

### 4.7 Test buttons preserved

The "Fill Test Data" / "Test" / "?preview" affordances live inside the .svelte components. They move with the components.

---

## 5. Critical invariants to preserve

These are the hidden contracts we must not break. Each is verifiable post-implementation.

1. **Status forward-only.** A cartridge or run must never transition backward through the lifecycle (except via abort).
2. **Write-once subdocs.** Every cartridge subdoc ( `backing` , `waxFilling` , `waxQc` , `waxStorage` , `ovenCure` , `reagentFilling` , `reagentInspection` , `topSeal` , `storage` ) is written exactly once, guarded by `{ recordedAt: { $exists: false } }` . The split must not introduce a path that re-enters a subdoc-writing action.
3. **Wax cooling gate.** `completeQC` rejects with HTTP 400 if `coolingConfirmedAt + 10 min > now` , unless admin bypass. Must still work after the move.
4. **Wax oven cure gate.** `scanBackingLot` rejects unless `elapsedMin >= minOvenTimeMin` , with admin override path. Stays in wax-filling step 2.
5. **Cartridge precondition for reagent.** `/api/dev/validate-equipment?type=cartridge&context=reagent` requires the cartridge to be in `wax_filled | wax_stored | wax_qc` . Unchanged.
6. **Robot mutual exclusion.** A robot must never simultaneously be running an active wax AND active reagent run. With Option B, this still holds because the new query filter only excludes post-OT-2 runs, not in-progress ones.
7. **Audit completeness.** Every status transition writes an AuditLog entry today. The actions don't change, so this is preserved.
8. **Part consumption integrity.** Every cartridge in `completeQC` triggers `recordTransaction(consumption)` ; every rejection triggers `recordTransaction(scrap)` ; counts must match. Unchanged.
9. **Cancel/abort during the OT-2 run only.** Available from steps 1-3 ( `Setup` , `Loading` , `Running` ) on the legacy wax-filling/reagent-filling pages. **Not available** once the OT-2 has completed successfully (i.e., once `robotReleasedAt` is set). The `cancelRun` / `abortRun` actions get a server-side guard: `if (run.robotReleasedAt) return fail(400, { error: 'Run can only be aborted before the robot finishes' })` . Individual-cartridge rejections ( `rejectCartridge` for wax, `rejectAtSeal` for reagent) remain available throughout — they void single cartridges, not runs.

---

## 6. Implementation plan (phased, lowest-risk first)

### Phase 0 — Branch hygiene (zero behavior change)

- Confirm we're on `wax-and-reagent-split-up` (we are).
- Commit the assay-shape work that's already in progress as a single coherent commit (so it doesn't tangle with the wax/reagent diff).

### Phase 1 — Add `robotReleasedAt` field, no behavior change

- Add `robotReleasedAt: Date` to `WaxFillingRun` and `ReagentBatchRecord` schemas.
- Wire `confirmDeckRemoved` (wax) and `completeRunFilling` (reagent) to set it.
- Existing queries don't read it yet — robot stays locked exactly as before.
- **Verification:** create a new wax run, complete steps 1-3, scan the deck removal, then check mongo: doc has both `status: 'Awaiting Removal'` and `robotReleasedAt: <timestamp>`. Robot lock unchanged.

### Phase 2 — Build the Opentron Control landing

- `src/routes/manufacturing/opentron-control/+page.server.ts` — load: robots + both queues. Reuses existing data shapes from `manufacturing/+page.server.ts:149-198` (the robot-status logic that's already correct).
- `src/routes/manufacturing/opentron-control/+page.svelte` — three sections (robots, wax queue, reagent queue). Marked as a frozen `.svelte` only after content is finalized — for now we are creating it, so it can be edited freely as a new file.
- Add nav entry between "Reagent Filling" and "QA/QC" in `manufacturing/+layout.svelte`.
- **Verification:** page loads, queues populate from existing in-progress runs, robot cards show correct availability.

### Phase 3 — Build the per-run post-OT-2 flow pages

- `opentron-control/wax/[runId]/+page.server.ts` — load: same data shape as the post-step-3 portion of wax-filling's load. Re-export the relevant actions (or import them and re-expose).
- `opentron-control/wax/[runId]/+page.svelte` — render the three components (`PostRunCooling`, `QCInspection`, `CompletionStorage`) the same way wax-filling does for those stages.
- Same for reagent: `opentron-control/reagent/[runId]/+page.server.ts` and `+page.svelte`.
- **Verification:** click a wax run from the queue, complete cooling → QC → storage. Check mongo: identical writes to the current flow (subdocs, status transitions, audit logs, transactions).

### Phase 4 — Flip the robot-release switch

- Modify the active-run queries (per §4.4) to add `robotReleasedAt: { $exists: false }`.
- This is the moment the robot is actually released early. After this commit, after a run hits split-point, the operator can start a new run on the same robot.
- **Verification:** start a wax run on Robot 1, complete through deck-removal. Robot 1 page no longer blocks new run creation. The post-step-3 cartridges still appear in the Opentron Control queue.

### Phase 5 — Redirect from old flow

- After the wax-filling step 3 → step 4 transition, redirect the operator to `/manufacturing/opentron-control/wax/[runId]` instead of staying on the wax-filling page.
- Same for reagent: after step 3 → step 4, redirect to `/manufacturing/opentron-control/reagent/[runId]`.
- The legacy wax-filling and reagent-filling pages can still render those steps if accessed directly (graceful fallback).

### Phase 6 — Cleanup (optional)

- Once Phase 5 has been live for a release cycle without issue, the post-step-3 rendering branches in wax-filling/+page.svelte and reagent-filling/+page.svelte could be tightened to redirect-only. But: those are frozen `.svelte` files — leave them alone. They simply become unreachable for those steps in normal navigation.

---

## 7. Open questions — answer before I start implementing

These are decisions only the user/team can make. I've flagged them in the relevant sections too.

### 7.1 Reagent top-seal deadline

The brief says: "for reagent, it should have the stop watch showing that they need to be top sealed before a specified time and then be stored."

**Today this deadline does not exist in the code.** I searched; there's no "must seal within X minutes" enforcement on filled cartridges. The closest analog is the cooling-queue page that uses `minCoolingTimeMin`, but that's a *minimum* time, not a *deadline*.

**What needs to be decided:**

1. **The deadline value.** How many minutes after the OT-2 finishes ( `runEndTime` ) is sealing considered "late"? This is a chemistry/process question — how long can filled reagent sit unsealed before the assay quality degrades? Examples: 30 min? 2 hours? 8 hours? Whatever the answer, we add a `maxTimeBeforeSealMin` setting to `ManufacturingSettings.reagentFilling` (currently has `fillTimePerCartridgeMin` and `minCoolingTimeMin` ).
2. **What happens when the deadline passes.** Two choices:
   - **Block** — once the timer hits zero, the "Complete Seal Batch" button greys out; sealing requires an admin password override (same mechanism as the wax cooling-bypass). Forces explicit acknowledgement that something went wrong.
   - **Warn** — big red countdown turns into a big red warning ("OVERDUE — seal immediately"), but the sealing actions still work. Operator's call.
3. **What the UI shows.** A countdown timer on each row of the "Reagent Cartridges Requiring Top Sealing and Storage" queue, plus a prominent timer on the per-run TopSealing page. Color-coded: green if plenty of time, yellow as the deadline nears, red once past.

**My recommendation: Warn**, not block. Reasons:

- Blocking risks the operator scrapping a perfectly-good batch because they were 5 seconds over a somewhat-arbitrary deadline.
- The override path adds friction without preventing the actual harm (degraded reagent — already happened by then).
- Warning + audit-log entry on overdue seals gives QA the data to evaluate whether the deadline is too aggressive or whether overruns correlate with failed assays later.
- If chemistry data later shows a sharp cliff at some duration, we can add the block then.

But if your chemistry team says "absolutely no sealing past N minutes," then block-with-override is the safer call.

### 7.2 Concurrent post-OT-2 runs — should there be a cap?

**The new situation.** Today a robot can have at most one wax or reagent run in flight at a time. After the split, the same robot can have multiple runs at different stages simultaneously, e.g.:

- Run #1: in QC on Opentron Control (waiting for inspection)
- Run #2: in QC on Opentron Control (waiting for storage)
- Run #3: just finished on the OT-2, robot now free, sitting in the wax queue
- Run #4: actively running on the OT-2

The runs themselves don't conflict — each one tracks its own cartridges, oven location, cooling tray, fridge assignment. But the *operator* now has multiple things to juggle.

**Where caps could be applied:**

- **No cap (default).** Queue rows just keep accumulating. Operator works through them in order. Many manufacturing shops do exactly this — let queue depth be its own feedback loop.
- **Per-operator cap.** Block starting a new run if the operator already has N runs awaiting post-processing. Forces them to clear the backlog first.
- **Per-fridge cap.** Block once a fridge is full or near-full of pending storage assignments.
- **Per-robot cap.** Block starting a new run on a robot if the previous run from that robot still hasn't been stored.

**My recommendation: no cap, but make the queue prominent.** The sections "Wax Cartridges Requiring Inspection and Storage" and "Reagent Cartridges Requiring Top Sealing and Storage" should be visually impossible to miss on the Opentron Control page — sorted oldest-first, with a count badge next to the section header, and rows that turn yellow/red as their cooling/seal deadlines approach. That makes "I have 6 things waiting" obvious at a glance, which is usually enough.

If after a few weeks you see operators consistently letting queues balloon, we can add a per-operator soft cap (warning at 4, hard block at 8, for example) without much effort. Easier to add later than to remove later.

### 7.3 Cancel/abort during post-OT-2 — RESOLVED

**Decision:** A run cannot be cancelled or aborted once it has completed successfully on the OT-2. Abort is only available during steps 1-3 ( `Setup` , `Loading` , `Running` ). After the robot finishes (i.e., once `robotReleasedAt` is set), the run is committed.

**Implementation:**

- The `cancelRun` and `abortRun` server actions get a guard: reject if `robotReleasedAt` exists on the run.
- These actions are not exposed on the new Opentron Control routes.
- Individual-cartridge rejections ( `rejectCartridge` during wax QC, `rejectAtSeal` during reagent sealing) remain available throughout the post-OT-2 flow — they void single cartridges, not whole runs.

### 7.4 Naming — RESOLVED

- Tab label: **"Opentron Control"**
- Route slug: **`/manufacturing/opentron-control`** (matches the existing convention of full-name slugs like `/manufacturing/wax-filling` )
- Section headers on the page:
  - "Wax Cartridges Requiring Inspection and Storage"
  - "Reagent Cartridges Requiring Top Sealing and Storage"

### 7.5 Existing "Wax Filling" / "Reagent Filling" tabs — RESOLVED

**Decision: Collapse both into Opentron Control.** The standalone `/manufacturing/wax-filling` and `/manufacturing/reagent-filling` nav entries get removed. Starting a new wax or reagent run happens by

clicking the appropriate button on a robot card on the Opentron Control page.

**Implementation note:** the underlying routes ( `/manufacturing/wax-filling/...` and `/manufacturing/reagent-filling/...` ) can stay alive as URL targets — they're where the actual step 1-3 UI lives — but they're reached by clicking from Opentron Control's robot cards, not from the sidebar. If we want to be tidy, we could rename them to `/manufacturing/opentron-control/wax/new?robot=X` and `/manufacturing/opentron-control/reagent/new?robot=X` , but that's cosmetic — keeping the existing routes accessible avoids breaking any bookmarks or in-flight runs.

---

## 8. Risk register

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Active-run query missed in some place, robot still locked | Med | Med | Grep for the query patterns — there are exactly 6 places (§4.4); audit each. |
| Cartridge subdoc gets re-written if action invoked twice | Low | High | All subdoc writes are guarded with `{recordedAt: {$exists: false}}` filters today. Verify those guards remain in the moved actions. |
| Two robots pick up the "same" backing lot from the queue | Low | High | Backing lots have a `status: 'in_oven' |
| Audit trail disconnected — operator sees post-processing without seeing the run that produced it | Med | Low | Queue rows + per-run pages all reference the underlying runId. AuditLog records survive unchanged. |
| Operator orphans a run (starts steps 1-3, never completes 4-6) | Med | Low | Already possible today. Queue surfaces them, so visibility actually improves. |
| Permission check missed on new routes | Med | Med | Reuse `requirePermission(locals.user, 'manufacturing:write')` exactly as the source routes do. |
| Real-time updates (timer countdowns) don't refresh after navigation away | Low | Low | Each `.svelte` component already manages its own intervals; they restart on remount. |

---

## 9. Summary of what does NOT change

To make explicit: every one of these is preserved as-is.

- Mongo schemas (other than the new `robotReleasedAt` field, additive only)
- Cartridge lifecycle status values
- WaxFillingRun status enum
- ReagentBatchRecord status enum
- Every `AuditLog.create` call — same tableName, action, recordId, newData
- Every `recordTransaction` call — same partDefinitionId, quantity, manufacturingStep, scrapReason, scrapCategory

- Equipment validation endpoints ( `/api/dev/validate-equipment` )
- Admin override mechanisms (cooling bypass via admin password, oven-time override via username+password)
- `/api/dev/test-data` endpoints used by Test buttons in `WaxPreparation` , `DeckLoadingGrid` , `ReagentPreparation`
- Cooling-queue page (still works, may also be linked from Opentron Control)
- Oven-queue page (still works)
- Equipment page (still works)
- Settings pages (still works)
- All `requirePermission` checks
- BackingLot lifecycle and oven-time gating
- Cartridge → BackingLot → Equipment lookup chain

---

## 10. Final decisions (all resolved — implementation can proceed)

1. ~~§3.3 Option B vs A~~ — **Answered: Option B** (add `robotReleasedAt` timestamp; no status enum changes).
2. ~~§7.1 top-seal deadline~~ — **Answered: warn-only, defaulting to 60 min.** Stored as `ManufacturingSettings.reagentFilling.maxTimeBeforeSealMin` ; surfaced on the reagent settings page so chemistry can adjust without code changes. Overdue seals get visual warnings + audit-log entries; sealing still works.
3. ~~§7.5 wax-filling/reagent-filling tabs~~ — **Answered: collapse into Opentron Control.**
4. ~~§7.4 route slug~~ — **Answered: `/manufacturing/opentron-control` .**
5. ~~§7.2 concurrency cap~~ — **Answered: no cap.** Queue sections will be visually prominent (count badges, color-coded time-pressure rows).
6. ~~§7.3 cancel/abort during post-OT-2~~ — **Answered: no abort once OT-2 completes.**

Implementation starts at §6 Phase 1.